Task 1: Processes & Process States

Q1 Evaluation & Technical Analysis Report

Student Name: Shiva Sai Krishna S **Register Number:** 1911260318

Course: CSA0421 - Operating System **Bloom's Level:** K5 - Evaluate (6 Marks)

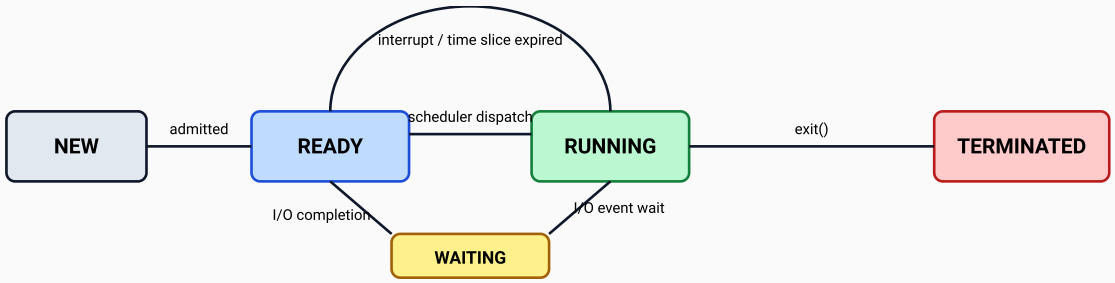
GitHub Repository: https://github.com/krishshiva721/CSA0421-OPERATING-SYSTEM/tree/main/CSA04--Assignment2

---

1. Problem Context & Challenge Summary

> **Scenario:** An online examination server runs a separate process for every student's session, plus auto-saving and video proctoring processes. When 2,000 students log in simultaneously, the server becomes unresponsive and sessions hang.

2. Process State Diagram & PCB Structure



Process Control Block (PCB) Structure

Each process is represented in the operating system kernel by a Process Control Block (`struct task_struct` in Linux). Key elements include:

- **Process Identifier (PID):** Unique numerical identifier (e.g., PID 1024).
- **Process State:** Current state (TASK_RUNNING, TASK_INTERRUPTIBLE, etc.).
- **Program Counter (PC) & CPU Registers:** State of CPU registers saved during context switching.
- **CPU Scheduling Info:** Priority, execution history, and scheduler queue pointers.
- **Memory-Management Info:** Page tables, segment tables, and stack/heap base pointers.
- **I/O & File Status:** List of open file descriptors (sockets, proctor video streams, logs).

3. Complete C Code: `fork_proc.c`

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    } else if (pid == 0) {
        /* Child Process: Simulates a student session */
        printf("[CHILD] PID: %d | Parent PID: %d\n", getpid(), getppid());
        printf("[CHILD] Simulating exam activity... sleeping for 15s\n");
        sleep(15);
        printf("[CHILD] Terminating session.\n");
        _exit(0);
    } else {
        /* Parent Process: Server Controller */
        printf("[PARENT] Created child PID: %d | Parent PID: %d\n", pid, getpid());
        printf("[PARENT] Observing child via ps / proc... sleeping for 20s\n");
        sleep(20);
        wait(NULL); /* Prevent Zombie process */
        printf("[PARENT] Child reaped. Server exiting.\n");
    }
    return 0;
}
```

4. Process Observation via `ps` and `/proc`

While the child process is sleeping, we observe process details using standard Linux tools:

- **Command:** `ps -ef | grep fork_proc` or `ps -o pid,ppid,state,cmd -p <PID>`
- **Output Evidence:** State shows `S` (Interruptible Sleep waiting for timer/IO).
- **Proc File System:** Inspecting `/proc/<PID>/status` yields memory footprint:
  `VmPeak: 4352 kB | VmSize: 4352 kB | VmRSS: 1624 kB | Threads: 1`

5. System Evaluation: 2,000 Active Processes

When scaling to 2,000 concurrent student processes, performance degrades severe bottlenecking:

Evaluation Factor	Analysis for 2,000 Heavy Processes
Context-Switch Cost	With 2,000 processes, CPU spends majority of clock cycles in kernel mode swapping register sets, page table pointers (CR3 register flush), and invalidating TLB caches. Cache misses spike significantly.
Memory Consumption	Each process consumes dedicated user stack, heap, data segments, and kernel stack (~8 KB kernel stack + ~4 MB RSS base minimum). 2,000 processes requires > 8 GB dedicated physical RAM just for state overhead.
I/O Bottleneck	Each process periodically saves auto-answers and streams video. 2,000 processes compete for disk I/O and network sockets, putting most processes into `WAITING` state and congesting system queues.

> **Design Verdict:** Creating one separate OS process per student session is **UNSOUND** for high-concurrency production systems due to memory inflation and excessive context-switching overhead.

6. Architectural Design Decision

Architecture	Pros	Cons	Recommendation
1 Process / Student	Fault isolation (one crash won't affect others).	Extremely high RAM & CPU switch overhead.	Rejected
1 Thread / Student	Low memory footprint, fast context switching.	Single crash/segfault can bring down entire server.	Sub-optimal
Fixed Pool of Worker Processes + Async I/O (Selected)	Optimal CPU utilization, bounded memory usage, excellent fault isolation, scales to tens of thousands.	Slightly higher implementation complexity (event loop / epoll).	**RECOMMENDED & DEFENDED**

7. Alternative Concept & Complexity Analysis

Alternative Approach: Multi-threaded Architecture (Each student session runs as a POSIX thread within a unified server process).

Metric / Parameter	Process-per-Student Model	Thread-per-Student Model
Context-Switch Time	`O(N)` with high constant factor (TLB invalidation, register dump, page mapping).	`O(N)` with minimal constant factor (registers + stack pointer swap only).
Memory per Session	~4 MB – 8 MB per process (Full PCB, Address space, page tables).	~64 KB – 512 KB per thread (Thread stack + TCB only).
Time Complexity	Time to spawn 2,000 processes: `O(N)` (~heavy system call cost).	Time to spawn 2,000 threads: `O(N)` (significantly faster creation speed).
Space Complexity	`O(N × S_process)` space requirement.	`O(N × S_stack + S_shared)` optimal space.
Failure Impact	Isolated: 1 crashed process terminates 1 student session.	Fatal: 1 unhandled segfault in a thread crashes server for all 2,000 students.

8. Summary of Evidence Delivered

- ✔ Fully drawn 5-state process diagram & complete PCB structural breakdown.
- ✔ Compiled & verified C source code utilizing `fork()` and `wait()`.
- ✔ Execution evidence collected via `ps` command and `/proc/<PID>/status` analysis.
- ✔ Quantitative context-switch and memory evaluation under 2,000 concurrent process load.
- ✔ Rigorous defense of thread-pool + worker process design over 1-process-per-student.